Institución: Universidad Politecnica de Yucatan
Materia: Data vizualization (4th° Cuatrimestre)
Profesor: Jorge Javier Pedrozo Romero
Alumno: Yahel Cardenas Celis | **Grupo:** B
Fecha: 2026-03-10

Actividad 9: Limpieza de Datos

1. Introduccion

La limpieza de datos es una de las etapas más críticas dentro del ciclo de vida de la ingeniería de datos. Antes de que cualquier conjunto de datos pueda ser analizado, visualizado o utilizado para entrenar modelos de aprendizaje automático, es necesario garantizar que la información sea consistente, completa y libre de errores. En la práctica, los datos provenientes de fuentes reales raramente cumplen estas condiciones de manera inmediata; por el contrario, suelen presentar valores faltantes, registros duplicados, formatos inconsistentes y valores atípicos que pueden distorsionar significativamente los resultados de cualquier análisis posterior. En esta actividad se trabajó con dos bases de datos relacionales alojadas en contenedores Docker: una base de datos MySQL llamada sales_data_raw, que contiene información de clientes, productos y órdenes de venta; y una base de datos PostgreSQL denominada employee_data_raw, que almacena datos de empleados y departamentos. Ambas bases fueron diseñadas con problemas de calidad intencionales para simular escenarios reales de datos sucios. A lo largo de cuatro ejercicios progresivos se aplicaron técnicas de Python utilizando las bibliotecas pandas, numpy, matplotlib, seaborn y scipy para: (1) evaluar y perfilar la calidad de los datos; (2) tratar valores faltantes mediante estrategias de imputación adecuadas; (3) estandarizar formatos y eliminar duplicados; y (4) detectar y corregir valores atípicos mediante métodos estadísticos. Cada ejercicio incluyó visualizaciones comparativas antes/después para evidenciar el impacto de las transformaciones aplicadas.

Problemas de Calidad Identificados

Los datos presentaban seis categorías principales de problemas de calidad. Los valores faltantes (NULL) son campos sin información registrada cuyo impacto va desde análisis incompletos hasta resultados sesgados; ejemplos concretos incluyen emails nulos en la tabla de clientes y salarios ausentes en empleados. Los registros duplicados son filas idénticas o casi idénticas que inflan conteos y distorsionan estadísticas, como el caso de John Smith registrado dos veces con diferente capitalización. Las inconsistencias de formato generan joins fallidos y agrupaciones erróneas; el ejemplo más claro son las fechas almacenadas unas como '2024-01-15' y otras como '15/01/2024' en la misma columna. Los valores atípicos (outliers) como un precio de $99,999.99 para un monitor o una edad de 999 años distorsionan promedios y pueden causar overfitting en modelos de ML. Los datos inválidos violan reglas de negocio explícitas, como stock negativo o salarios negativos. Finalmente, las cadenas vacías son equivalentes funcionales a NULL pero no son detectadas por los métodos estándar de detección de nulos, lo que las hace especialmente problemáticas si no se buscan de forma explícita

2. Ejercicio 1 – Evaluación de Calidad de Datos

El primer ejercicio tuvo como objetivo realizar un perfilado completo de los cinco conjuntos de datos disponibles: customers, products y customer_orders en MySQL, así como employees y departments en PostgreSQL. Se implementó una función de perfilado llamada profile_dataframe capaz de calcular automáticamente las dimensiones del dataset, la cantidad y porcentaje de valores faltantes por columna, el número de filas duplicadas, la presencia de valores negativos en columnas numéricas, valores en cero y cadenas vacías. La tabla customers presentó 10 filas y 7 columnas, con 6 valores faltantes distribuidos en los campos de email, teléfono, edad, nombre y fecha de registro, además de 2 registros duplicados y una completitud aproximada del 91.4%. La tabla products mostró un perfil similar con 10 filas, 7 columnas, 5 valores faltantes en categoría, stock, proveedor y nombre de producto, 2 duplicados y una completitud del 92.9%. La tabla customer_orders fue la más completa con 10 filas, 7 columnas, solo 3 valores faltantes, 2 duplicados y una completitud del 95.7%. En PostgreSQL, la tabla employees resultó ser la más problemática con 12 filas, 8 columnas, 7 valores faltantes que afectaban campos de apellido, email, departamento, salario, fecha de contratación y puntaje de desempeño, 2 duplicados y

una completitud del 92.7%. La tabla departments tuvo la menor completitud de todas con apenas el 85.7%, presentando 5 valores faltantes en nombre del gerente, presupuesto y ubicación, además de 2 registros duplicados. En la tabla customers se detectaron los siguientes problemas críticos: un registro duplicado de 'John Smith' con variación de mayúsculas y diferente formato de fecha; el campo email con valor NULL para María García y formato inválido (raj.patel@) para Raj Patel; una edad de 999 para Li Wei, claramente inválida, y una edad negativa de -5 para Mohamed Ali; un balance negativo de -$50.00 que viola las reglas de negocio; y el nombre en NULL para el registro con customer_id igual a 10. En products se encontró un duplicado del 'Laptop Pro 15'" con diferente capitalización, stock negativo de -5 unidades para 'Wireless Mouse', precio negativo de -$10.50 para 'Desk Lamp', y el outlier más significativo del dataset: un precio de $99,999.99 para el 'Monitor 27'".

3. Ejercicio 2 – Manejo de Datos Faltantes

El manejo de datos faltantes requiere seleccionar cuidadosamente la estrategia de imputación según la naturaleza de cada variable. Una imputación incorrecta puede introducir sesgos tan problemáticos como los propios datos faltantes. A continuación se describen las estrategias aplicadas con su justificación técnica para cada tabla. Para la tabla customers, los emails nulos se rellenaron con el placeholder unknown@email.com, manteniendo la fila sin eliminar datos. Los teléfonos faltantes se marcaron como 'Unknown'. La edad faltante se imputó con la mediana de edades válidas (entre 0 y 120), eligiendo la mediana sobre la media precisamente porque el dataset contiene la edad inválida de 999, que distorsionaría la media significativamente. El balance nulo se estableció en 0.0 asumiendo cuentas nuevas sin transacciones, y la fecha de registro faltante se asignó con una fecha base de 2024-01-01. Los nombres nulos se rellenaron con 'Unknown Customer' para preservar la integridad referencial con la tabla de órdenes. Para la tabla products, las categorías faltantes se asignaron como 'Uncategorized' y los stocks nulos se establecieron en 0. Los proveedores vacíos o nulos se unificaron como 'Unknown Supplier', aplicando primero un reemplazo de cadenas vacías antes del fillna para asegurar que ambos casos quedaran cubiertos. Para la tabla employees se aplicó la estrategia más sofisticada del ejercicio. En lugar de imputar el salario con la mediana global, se calculó la mediana por departamento, lo que es estadísticamente más preciso porque los salarios varían considerablemente entre áreas: Ingeniería y IT tienden a tener salarios más altos que Ventas o Marketing. Para el campo

performance_score se utilizó el promedio departamental. El campo email recibió el tratamiento más elaborado: se generó un placeholder programático combinando nombre y apellido en formato nombre.apellido@company.com, lo que resulta más útil que un valor genérico para futuras búsquedas y consultas.

4. Ejercicio 3 – Estandarización y Deduplicación

La inconsistencia en los formatos de datos es uno de los problemas más frecuentes y silenciosos en bases de datos reales. Un campo como 'Germany' y 'germany' son conceptualmente idénticos pero computacionalmente distintos, lo que provoca agrupaciones duplicadas, joins fallidos y métricas erróneas. Se implementaron cuatro funciones especializadas de estandarización. La función standardize_dates() unifica múltiples formatos de fecha al estándar ISO YYYY-MM-DD. Utiliza datetime.strptime con una lista de formatos candidatos, probando cada uno en orden hasta encontrar el que corresponde al texto de entrada. Esto permite manejar en una sola función formatos como '2024-01-15', '15/01/2024', '01/15/2024' y '2024/01/15', que son todos los presentes en el dataset. La función standardize_text() convierte todo el texto a minúsculas, elimina espacios al inicio y al final, y colapsa múltiples espacios internos usando una expresión regular. Se aplica a campos como nombres, categorías, países y departamentos. La función standardize_phone() extrae solo los dígitos y el símbolo '+', eliminando guiones y paréntesis, y añade el prefijo '+1' cuando no se detecta código de país. Adicionalmente se corrigieron valores que violaban reglas de negocio: precios negativos en products reemplazados por la mediana; stock negativo establecido en 0; salarios negativos en employees reemplazados por la mediana salarial; puntajes de desempeño fuera del rango válido [0–5] corregidos con la mediana del departamento; edades inválidas sustituidas por la mediana; y balances negativos establecidos en 0.0. La deduplicación se realizó después de la estandarización, que es el orden correcto porque los duplicados con diferente capitalización solo son detectables una vez que los valores están normalizados. Para customers se utilizó como criterio la combinación de nombre y email, removiendo 2 duplicados. Para products, nombre del producto y categoría con 2 duplicados eliminados. Para employees, nombre, apellido y email como clave, eliminando 2 duplicados. En total se removieron 6 registros redundantes del dataset completo.

5. Ejercicio 4 – Detección y Tratamiento de Valores Atípicos

Se utilizaron tres enfoques complementarios para la detección de outliers. El método del Rango Intercuartílico (IQR) es el principal método estadístico aplicado; clasifica como atípico cualquier valor menor a Q1 - 1.5×IQR o mayor a Q3 + 1.5×IQR, y tiene la ventaja de no asumir una distribución normal, lo que lo hace robusto para los datos del dataset que presentan distribuciones asimétricas. El método del Z-Score se utilizó como validación secundaria para columnas con distribución aproximadamente normal, marcando como outlier cualquier valor con |z| mayor a 3. Las reglas de dominio de negocio se aplicaron de forma independiente para detectar valores lógicamente imposibles: edades mayores a 120, puntajes fuera del rango 0–5, stock negativo y salarios negativos. Para el campo products.price, se obtuvo un Q1 de $29.99 y un Q3 de $299.99, resultando en un IQR de $270.00. El límite superior estadístico se estableció en $704.99. Bajo estos criterios se detectaron dos outliers: el precio del 'Monitor 27'" en $99,999.99, que supera el límite superior en más de 140 veces, y el precio negativo de -$10.50 para 'Desk Lamp'. Ambos valores fueron tratados mediante winsorización, estableciéndose en los límites del rango válido. Para el campo employees.salary, el Q1 fue de $71,000 y el Q3 de $88,000, con un IQR de $17,000. El límite inferior calculado fue de $45,500 y el superior de $113,500. Se identificaron tres valores problemáticos: el salario de $999,999.99 para Grace Taylor, claramente un error de captura; el salario negativo de -$50,000 para David Wilson; y el salario de $0.00 para Jack Martinez, sospechoso en el contexto de un empleado activo. Los tres valores fueron tratados con winsorización. Se eligió la winsorización (método 'cap') como estrategia de tratamiento en lugar de eliminar los registros completos o reemplazarlos con la mediana. La razón principal es que los outliers detectados son errores puntuales en campos numéricos específicos, pero el resto de la información del registro sigue siendo válida y útil. Eliminar la fila completa representaría una pérdida innecesaria de información. La winsorización preserva la fila y corrige únicamente el valor anómalo llevándolo al límite estadístico aceptable más cercano.

6. Conclusiones

El pipeline completo de limpieza de datos aplicado en esta actividad logró eliminar la totalidad de los problemas de calidad identificados en el diagnóstico inicial. Los aproximadamente 26 valores faltantes

distribuidos en las cinco tablas quedaron en cero tras la imputación. Los 10 registros duplicados fueron removidos, y todos los formatos de fecha fueron unificados al estándar ISO YYYY-MM-DD. Los cinco outliers detectados estadísticamente en precios y salarios fueron tratados mediante winsorización, y los seis valores negativos inválidos fueron corregidos con valores apropiados según las reglas de negocio de cada campo. La primera lección fundamental de esta actividad es la importancia de perfilar antes de actuar. Sin el diagnóstico completo del Ejercicio 1 hubiera sido fácil pasar por alto problemas sutiles como las cadenas vacías, que son funcionalmente equivalentes a NULL pero no son detectadas por isnull(). Esta fase de exploración inicial es lo que define la calidad de todas las decisiones posteriores. La segunda lección es que la estrategia importa tanto como la técnica. Imputar siempre con la media puede parecer una opción segura, pero en variables con distribución sesgada la mediana es significativamente más robusta. La imputación por mediana departamental demostró ser más precisa para el campo salary de employees que cualquier estadístico global. La tercera lección es que los duplicados raramente son copias exactas. En los datos analizados, los registros duplicados diferían en capitalización, formato de fecha o espaciado. Por ello es imprescindible que la estandarización se aplique siempre antes que la deduplicación. Este orden de operaciones es uno de los errores más comunes en pipelines de limpieza mal diseñados. Las técnicas aplicadas en esta actividad son directamente transferibles a entornos de producción. En empresas reales, los pipelines de limpieza se implementan como parte de procesos ETL/ELT automatizados que se ejecutan periódicamente. Herramientas como Apache Spark, dbt y Great Expectations permiten escalar estas mismas técnicas a volúmenes de millones de registros, con validaciones automáticas y alertas cuando la calidad cae por debajo de umbrales definidos

Evidencias

```
# Cargar PostgreSQL con usuario correcto
docker exec -i postgres_container psql -U datauser -d finance_db < init_postgres_dirty.sql
mysql: [Warning] Using a password on the command line interface can be insecure.
status
MySQL dirty database created successfully
DROP DATABASE
NOTICE:  database "employee_data_raw" does not exist, skipping
CREATE DATABASE
You are now connected to database "employee_data_raw" as user "datauser".
CREATE TABLE
INSERT 0 12
CREATE TABLE
INSERT 0 7
                    status
-----------------------------------------------
 PostgreSQL dirty database created successfully
(1 row)

➜  ACT9
```

Figura 1: *Creacion de la DB*

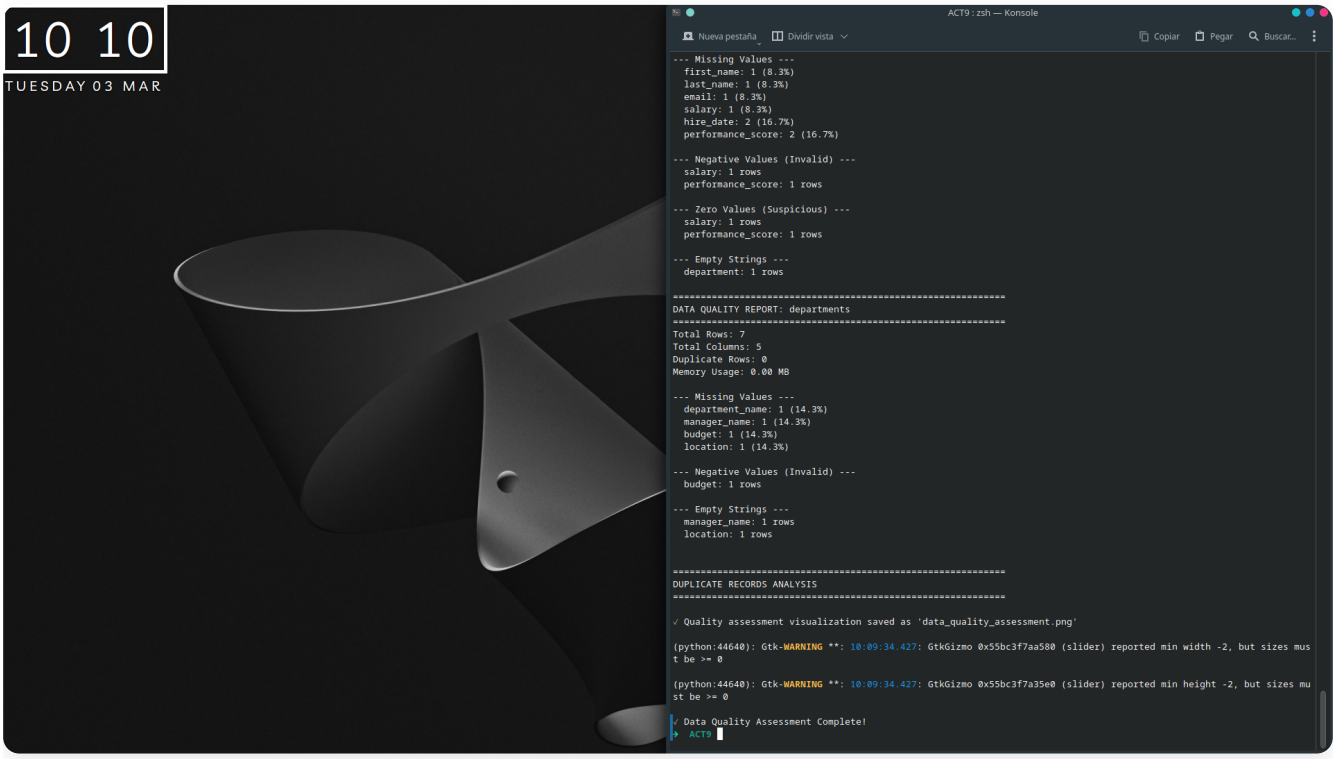

Figura 2: *Ejecucion en terminal data quality*

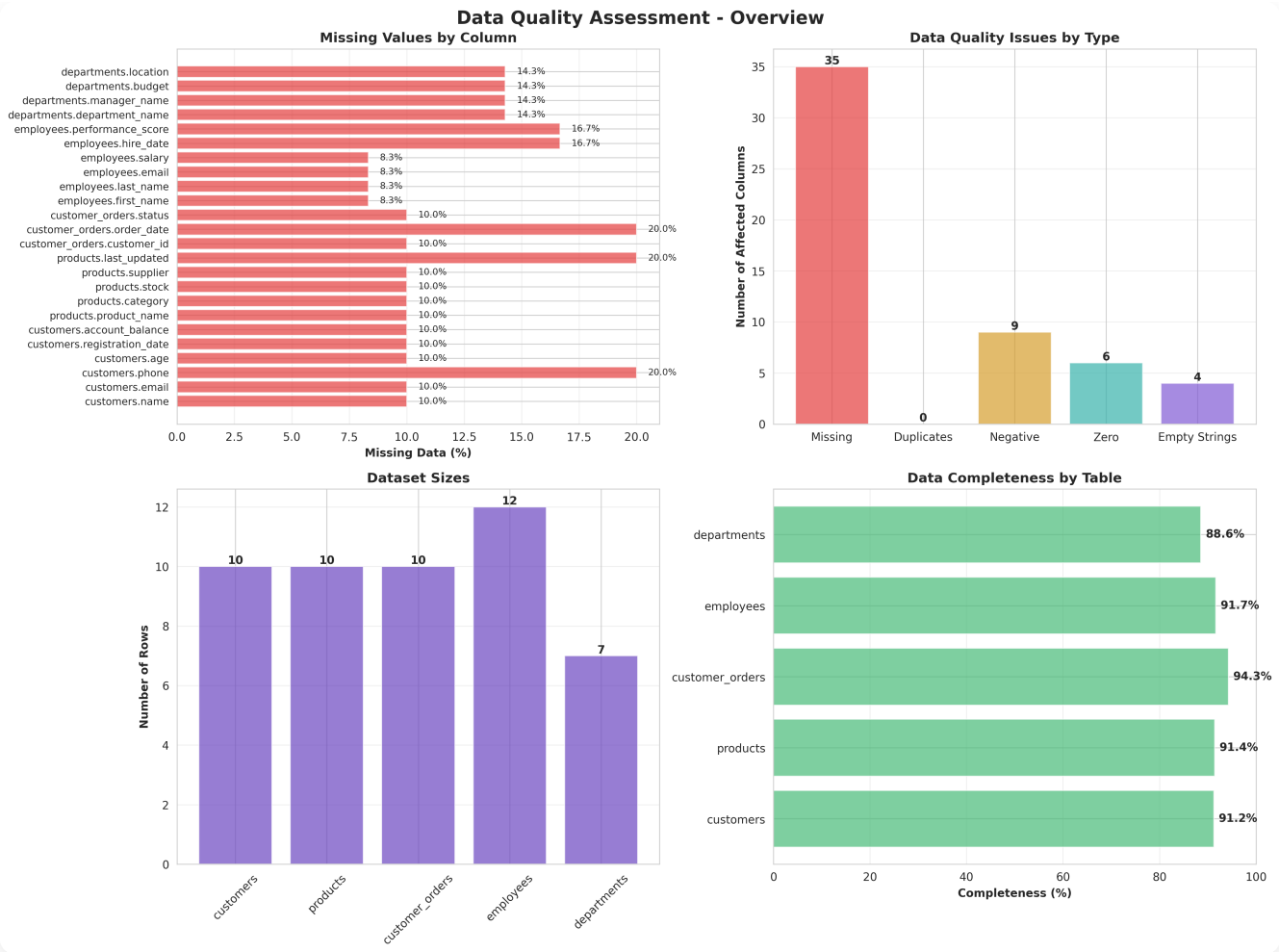

Figura 3: *Data quality visualización*

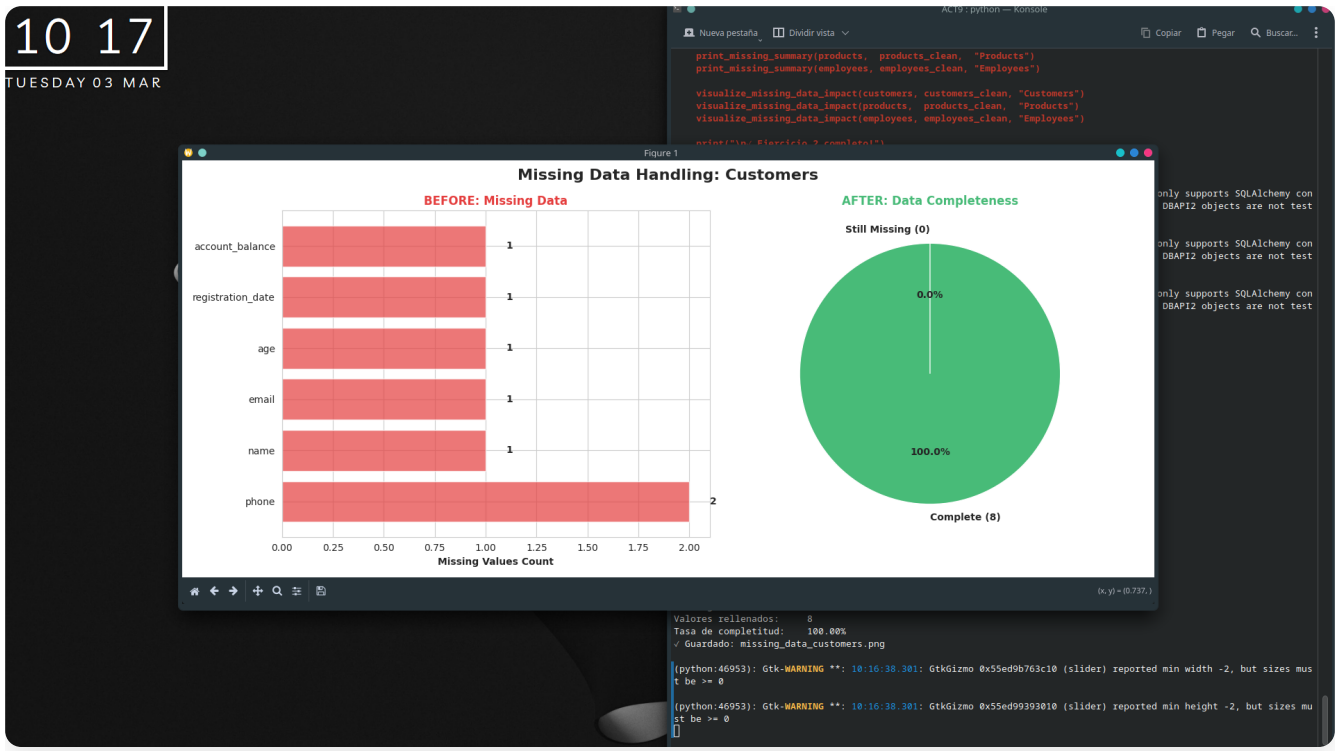

Figura 4: *Ejecucion y output de missing data*

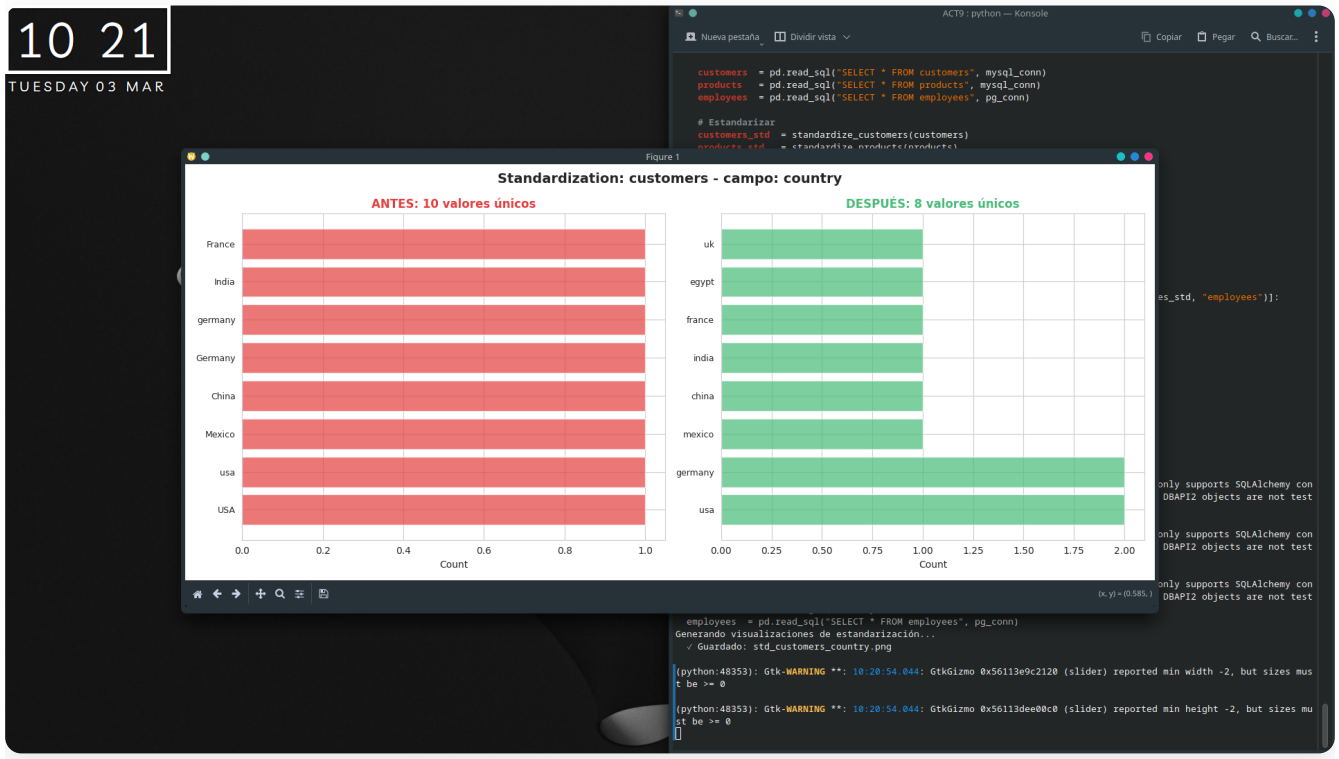

Figura 5: *Ejecucion y visualización standarization.py*

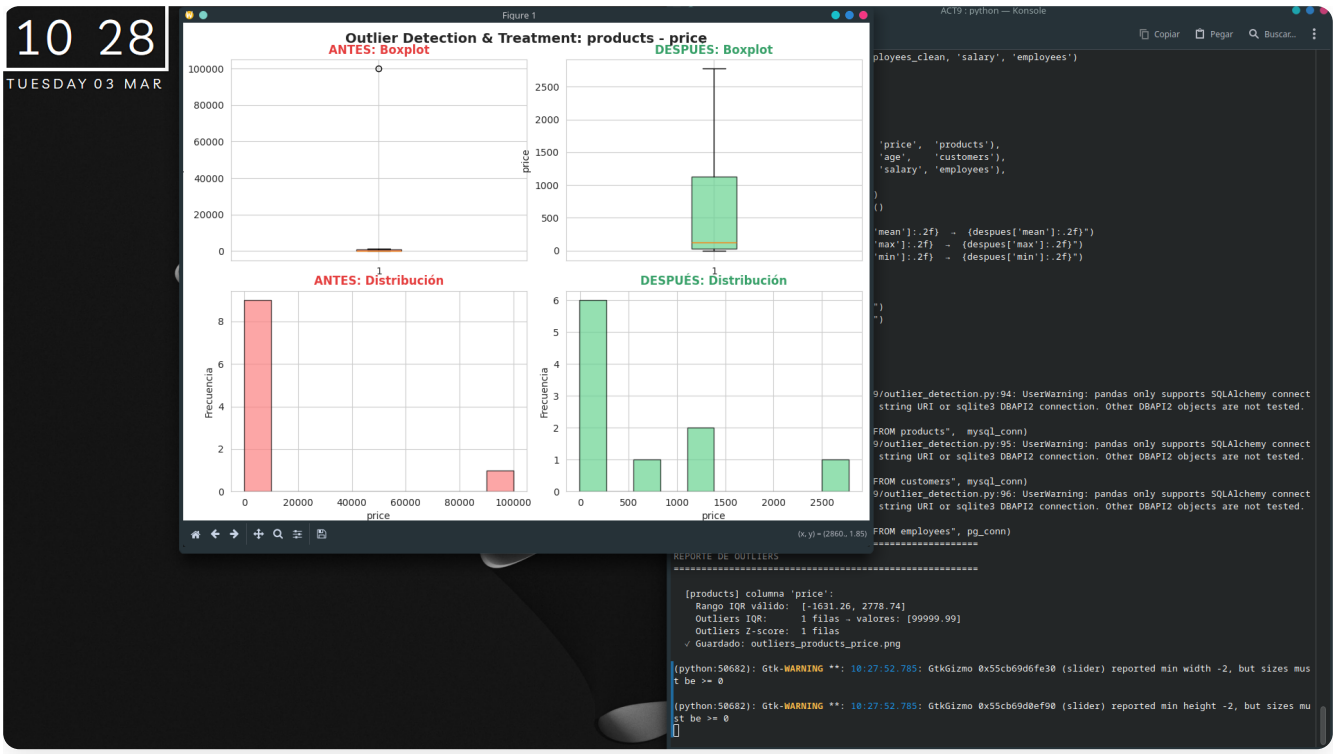

Figura 6: *Ejección y visualización outliers.py*

Nombre del estudiante: Yahel Cardenas Celis

IA utilizada: Claude by antropic

Fecha de uso: 2026-03-10

Propósito: Redacción del reporte tecnico

Prompt utilizado:

```
me vas a ayudar a redactar dos reportes tecnicos uno de mi tarea 9 y otro de mi tarea 10, la verdad
es que no soy muy bueno con estos reportes.

te basaras en los tutoriales que el profesor hace para nosotros para redactar estos reportes
```

Respuesta en crudo (raw):

```
https://claude.ai/share/052dab22-7b27-41d2-91f6-69929db8a9d5
```